# Don’t forget the attendance check!



# Testing (Practice)

Week 7-1

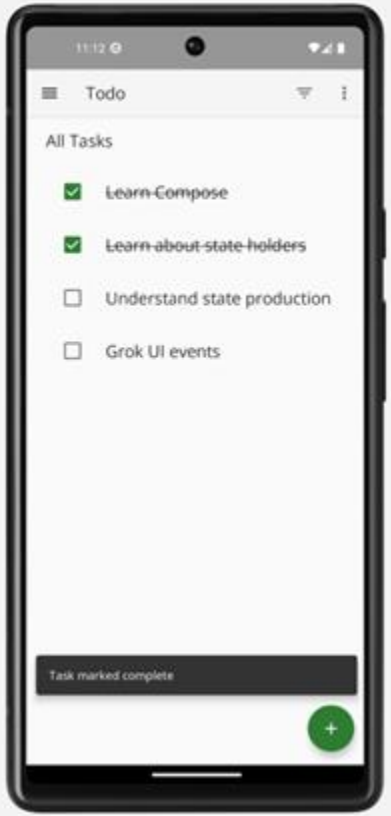
*Any fool can write code that a computer can understand. Good programmers write code that humans can understand.*

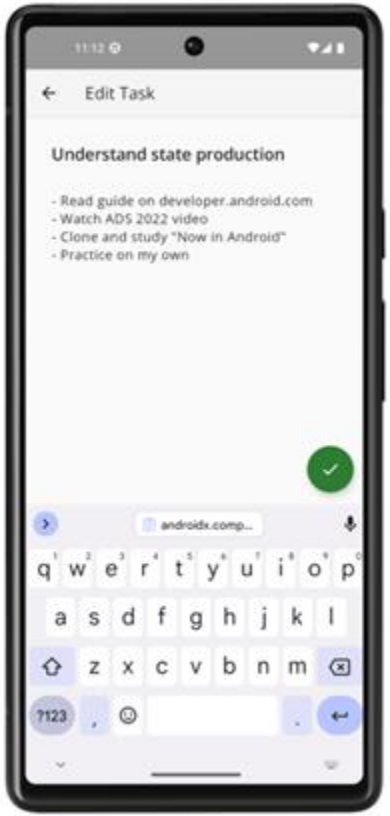
— Martin Fowler

# Objectives

- App Testing Practice using Todo app 😉
- Unit Tests in Two Layers:
  - **Model** Testing (Junit)
  - **ViewModel** Testing (Mockito + Hilt)

# Let’s test a Todo Application!

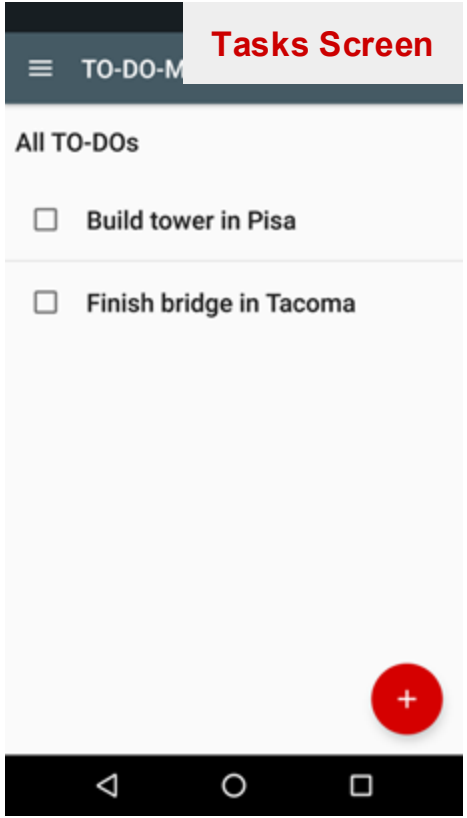



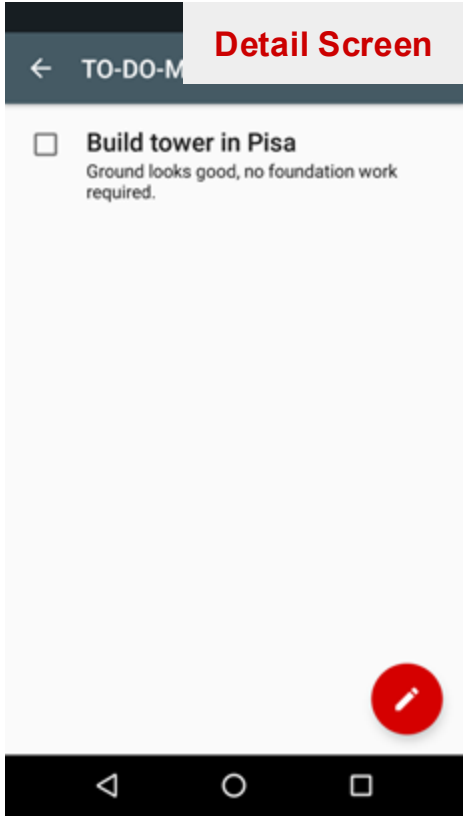



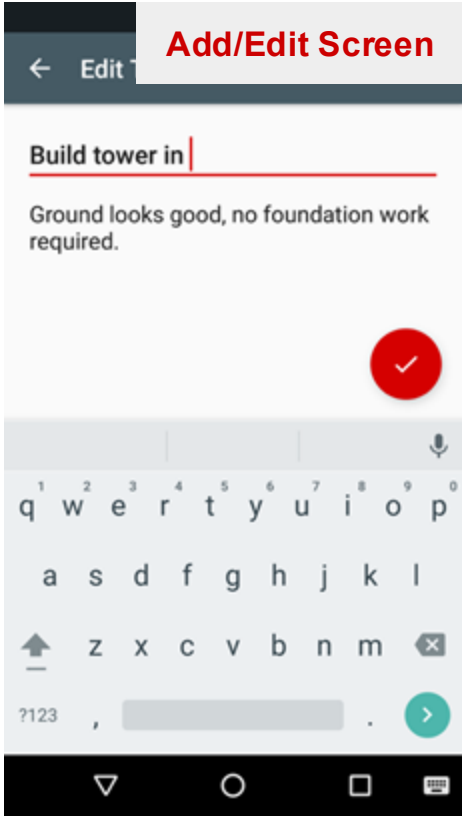
# App Specification

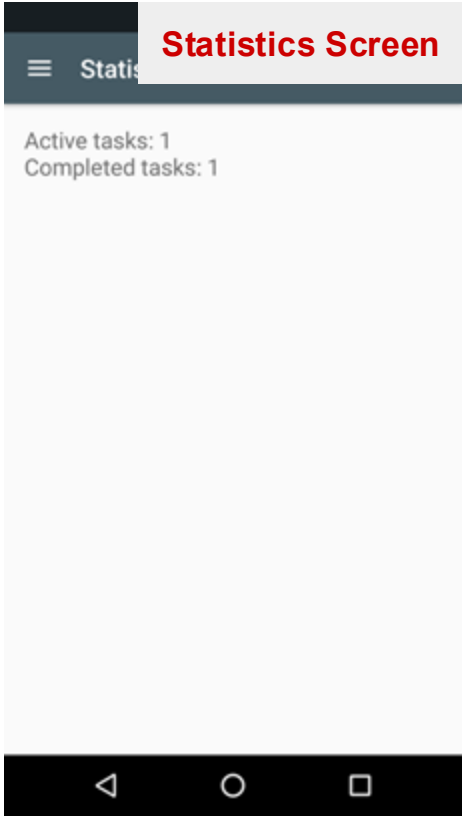
App skeleton

App specification



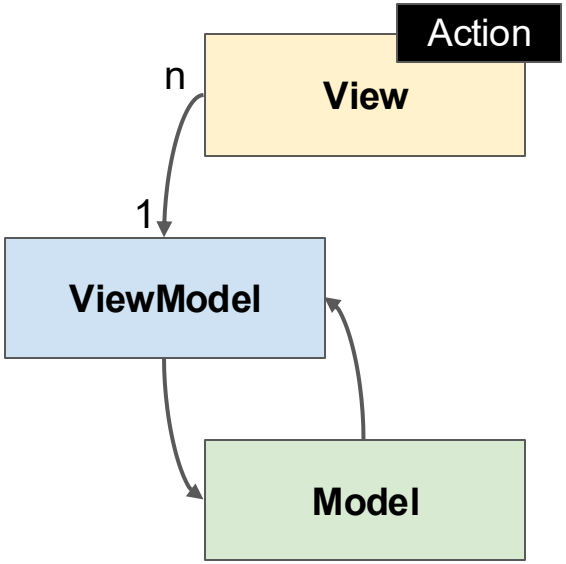
# Get the Skeleton Code

- Use Git
  - Clone the skeleton repository in the previous slide
- Or you could download the raw zip file

- **Submission Preparation**
  - Create a `submission` directory
  - Make subdirectories `exercise01 - exercise04`
  - You have to submit these files at the end of this week!

# Recap: MVVM

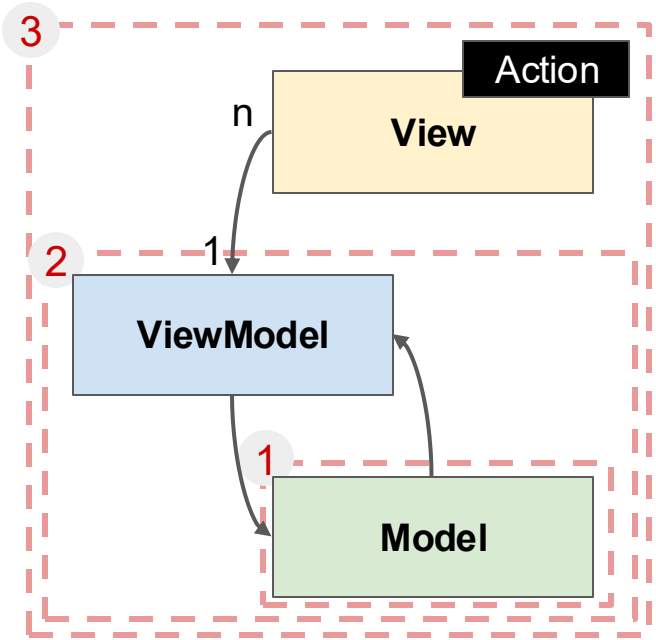
- An architectural design pattern used to separate concerns in UI applications
  - Low dependency between view, viewmodel, and model
    → easy to reuse!



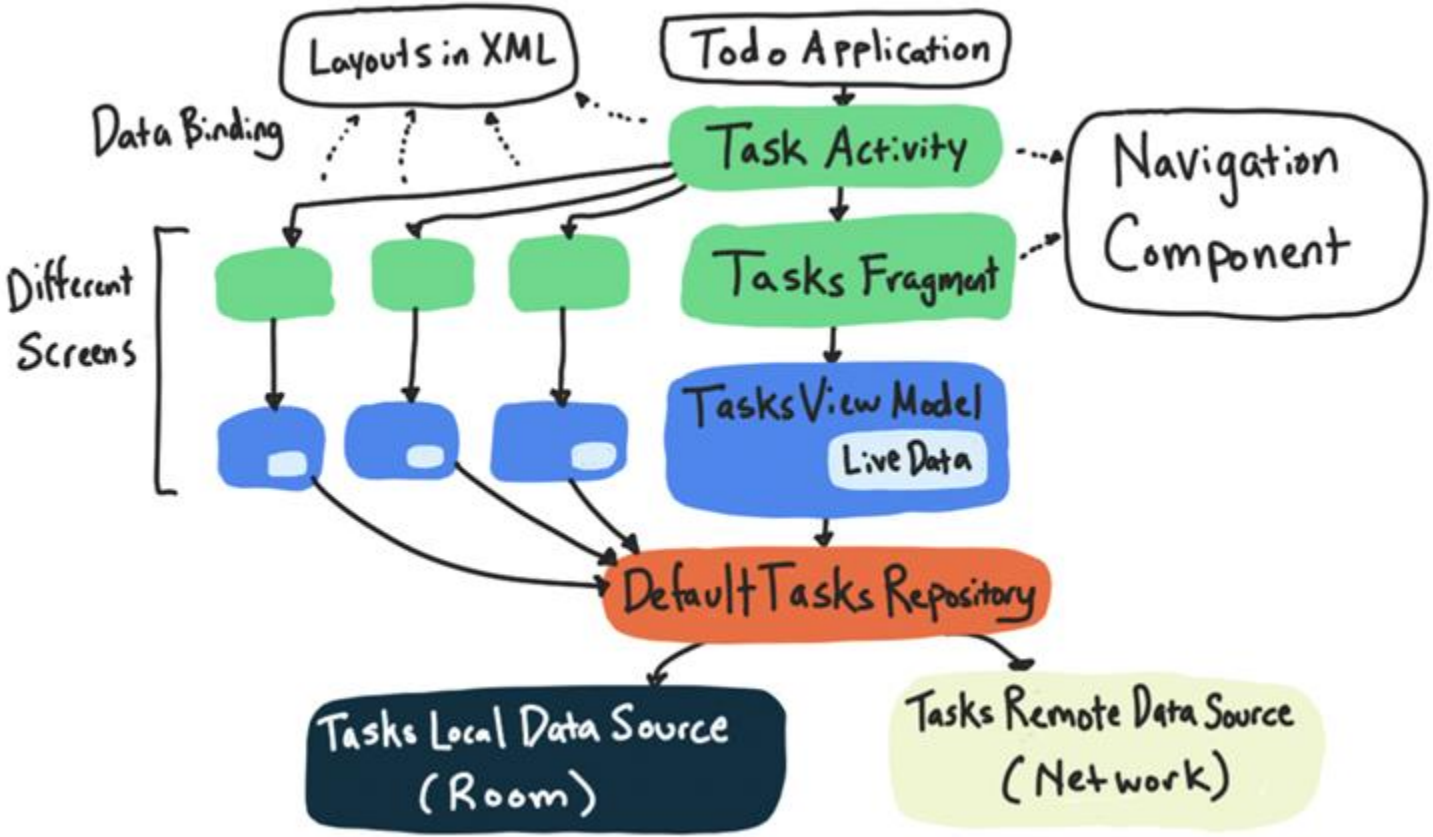
# Goal: Test considering architecture patterns

Testing Order:

**Model → ViewModel → View → Integrated Test**

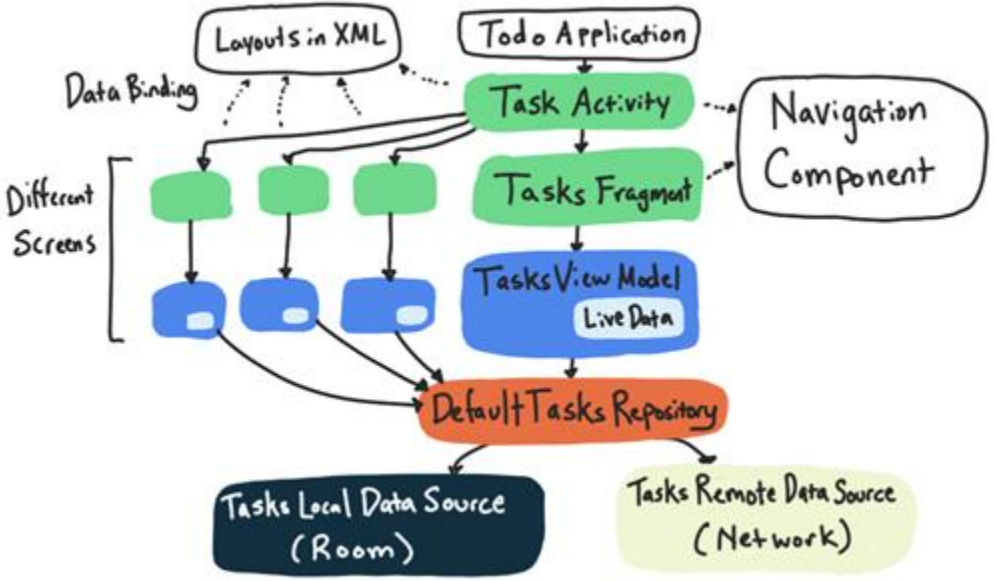


# App Architecture



# App Architecture

- Single activity architecture
- 1 view + 1 viewmodel per screen
- Data layer with a repository connected to 2 data sources



# Testing Strategy

1. First you'll **unit test** the model.
2. Then you'll use a **test double** in the view model, which is necessary for unit testing and integration testing the view model.
3. Next, you'll learn to write **integration tests** for fragments and their view models.
4. Finally, you'll learn to write **integration tests** that include the Navigation component.

https://developer.android.com/codelabs/advanced-android-kotlin-training-testing-test-doubles#2

# Testing Strategy

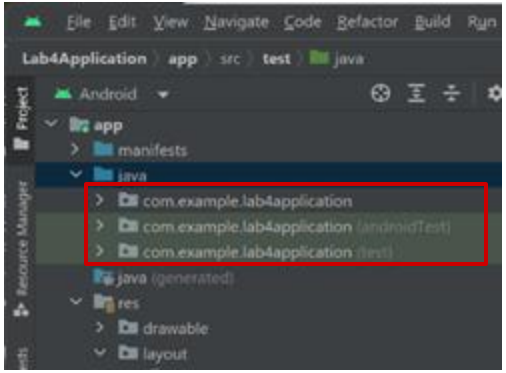
This week's scope!

1. First you'll **unit test** the model.
2. Then you'll use a **test double** in the view model, which is necessary for unit testing and integration testing the view model.
3. Next, you'll learn to write **integration tests** for fragments and their view models.
4. Finally, you'll learn to write **integration tests** that include the Navigation component.

https://developer.android.com/codelabs/advanced-android-kotlin-training-testing-test-doubles#2

# Android Studio: Testing Interface

# Test Source Sets




3 **source sets**:

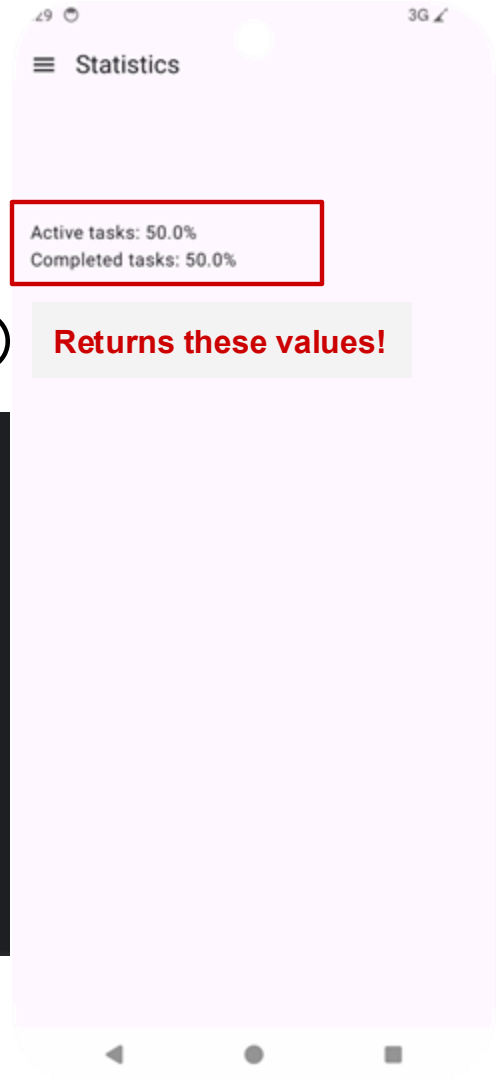
- `main`: contains app code
- `androidTest`: contains instrumented tests
  - Runs on real/emulated device
  - Slow
  - High fidelity
  - For integration, E2E, UI tests
- **`test`**: contains local tests
  - Runs on JVM (not on real/emulated device)
  - Fast
  - Low fidelity
  - For unit tests

# Writing Unit Tests (1/6)

- We'll write a unit test for `StatisticUtils.getActiveAndCompletedStats()`

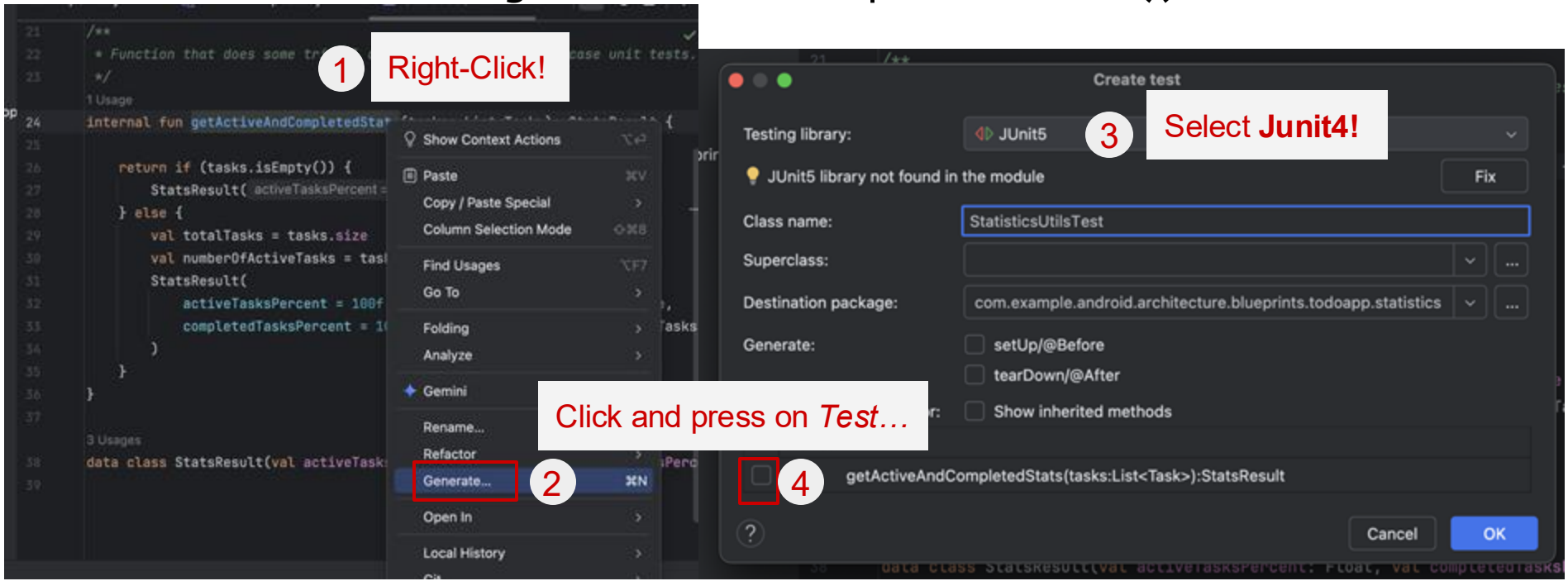
```
internal fun getActiveAndCompletedStats(tasks: List<Task>): StatsResult {

    return if (tasks.isEmpty()) {
        StatsResult( activeTasksPercent = 0f,  completedTasksPercent = 0f)
    } else {
        val totalTasks = tasks.size
        val numberOfActiveTasks = tasks.count { it.isActive }
        StatsResult(
            activeTasksPercent = 100f * numberOfActiveTasks / tasks.size,
            completedTasksPercent = 100f * (totalTasks - numberOfActiveTasks) / tasks.size
        )
    }
}
```



# Writing Unit Tests (2/6)

- We'll write a unit test for `StatisticUtils.getActiveAndCompletedStats()`



# Writing Unit Tests (3/6)

- Move the created Test file to `test` directory
  - Naming convention: `{filename}Test`

# Writing Unit Tests (4/6)

- If the input to `getActiveAndCompletedStats` is a task that consists of one task completed and one task active, it should return `StatsResult(0.5, 0.5)`. Let's test this!
- Add a function called `getActiveAndCompletedStats_two_return50f50f`
  - Naming convention used:
    - `subjectUnderTest_actionOrInput_resultState`

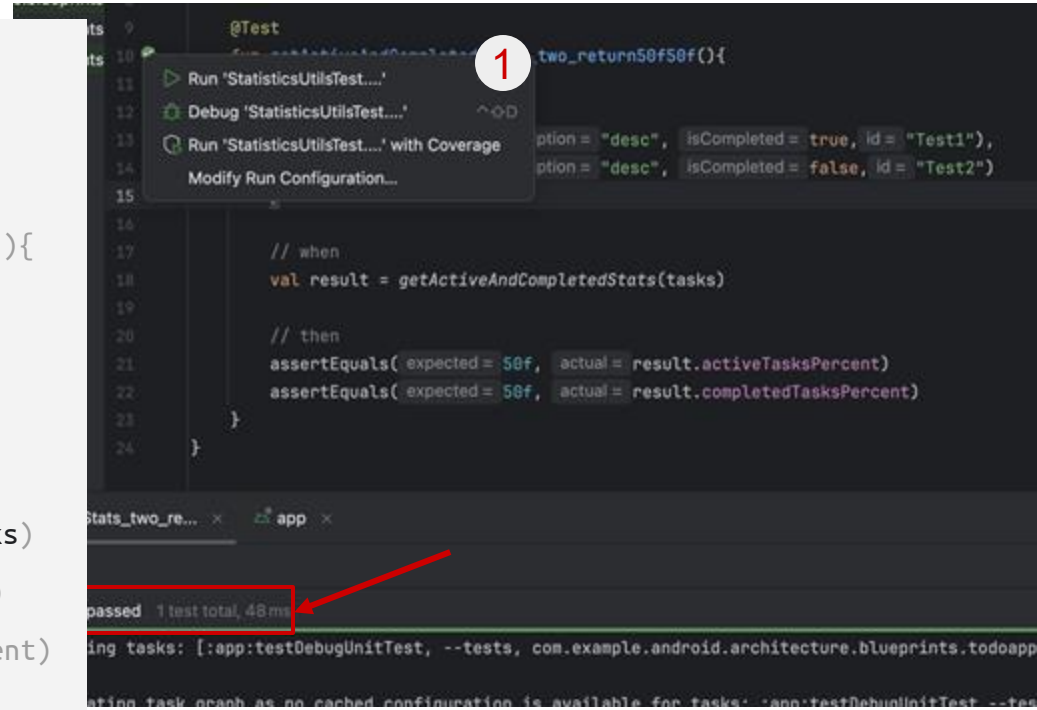
# Writing Unit Tests (5/6)

- Fill in using this testing mnemonic (import libraries as necessary):
  - Given / When / Then or AAA (Arrange, Act, Assert)
    - Given (Arrange): Setup the objects and app state for the test
    - When (Act): Do the actual action on the object
    - Then (Assert): Check what happens upon action

# Writing Unit Tests (6/6)

- Run your test with the green run button; it should pass

```
class StatisticsUtilsTest() {
    // ...
    @Test
    fun getActiveAndCompletedStats_two_return50f50f(){
        val tasks = listOf(
            Task("t1", "desc", true,”Test1”),
            Task("t2", "desc", false,”Test2”)
        )
        val result = getActiveAndCompletedStats(tasks)
        assertEquals(50f, result.activeTasksPercent)
        assertEquals(50f, result.completedTasksPercent)
    }
```



3 mins

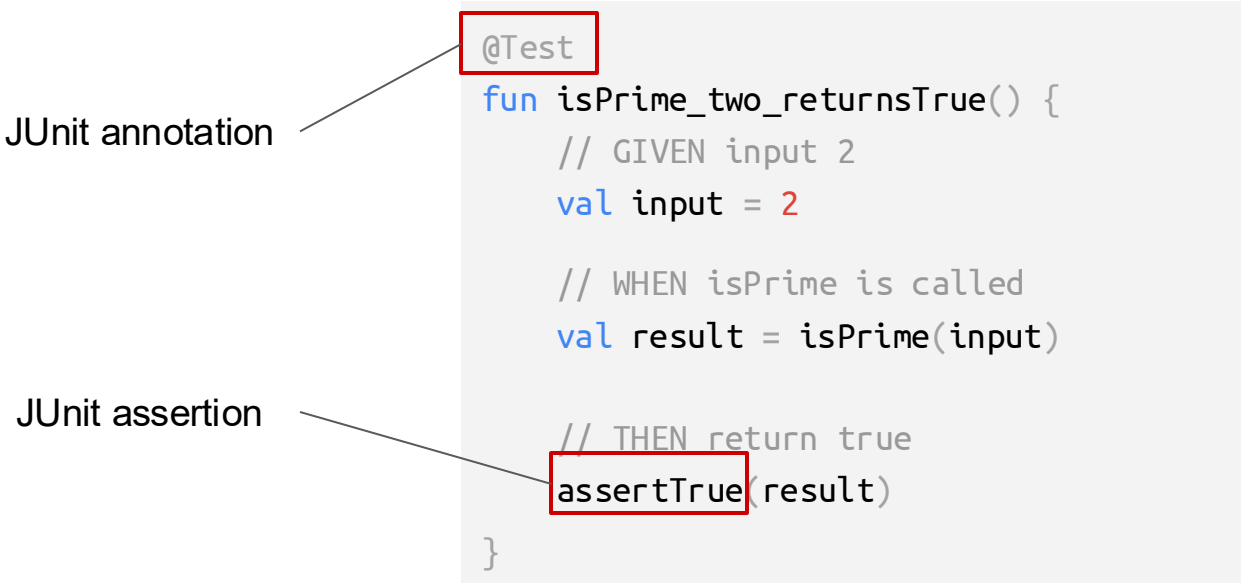
# Exercise 1: Write your own test

- Make 2 more tests that return different inputs/expected outputs
- and run all tests to check if it succeeds!

# Unit Test: JUnit Basics

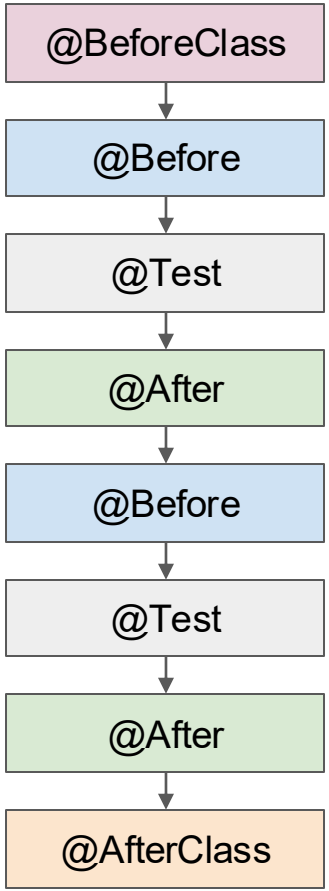
# JUnit4

- Unit testing framework for Java and Kotlin (family of xUnit)
- Used to write and run repeatable automated unit tests
- Requires Java 6 (or higher)



# JUnit Annotations

| Annotation | Meaning |
|---|---|
| `@Test` | Denotes that a method is a test method |
| `@Before` | Denotes that the annotated method should be executed *before* **each** test method in the current class |
| `@After` | Denotes that the annotated method should be executed *after* **each** test method in the current class |
| `@BeforeClass` | Denotes that the annotated method should be executed *before* **all** test method in the current class. The annotated methods must be static |
| `@AfterClass` | Denotes that the annotated method should be executed *after* **all** test method in the current class. The annotated methods must be static |
| `@Ignore` | Used to disable a test class or test method |

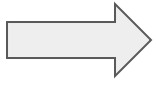



Source: https://junit.org/junit4/javadoc/4.12/org/junit/package-summary.html

# JUnit Annotations: Example

- Avoid redundant code during test setup

```
class MyTest {
    @Test
    fun test1() {
        val myObject = MyClass()
        // test something
    }
    @Test
    fun test2() {
        val myObject = MyClass()
        // test something
    }
    @Test
    fun test3() {
        val myObject = MyClass()
        // test something
    }
}
```



```
class MyTest {
    private lateinit var myObject: MyClass
    @Before
    fun setup(){
        myObject = MyClass()
    }
    @Test
    fun test1() {
        // test something
    }
    @Test
    fun test2() {
        // test something
    }
    @Test
    fun test3() {
        // test something
    }
}
```

# JUnit Assertions

| Assertions | Meaning |
|---|---|
| `assertAll` | Asserts that all supplied executables do not throw exceptions |
| `assertEquals / assertNotEquals` | Asserts that expected and actual are (not) equal |
| `assertFalse / assertTrue` | Asserts that the supplied condition is false / true |
| `assertNull / assertNotNull` | Asserts that the supplied object is (not) null |
| `assertThrows` | Asserts that the executable throws an exception |
| `fail` | Fails a test |
| `assertTimeout / assertTimeoutPreemptively` | Asserts that the executable completes before the given timeout is exceeded (preempts if exceed) |

Source: https://junit.org/junit4/javadoc/4.12/org/junit/Assert.html

# 3rd-Party Assertion Libraries

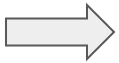
| 3rd Party Assertion Library | Description |
|---|---|
| AssertJ | Assertion library in Java to enhance readability |
| Hamcrest | To write more readable matchers |
| Truth | Modern Google assertion library |

```
import org.junit.Test

import org.junit.Assert.*

class JUnitAssertionDemo {

    @Test
    fun assertWithoutHamcrestMatcher() {
        assertEquals(3, sum(2 + 1))
    }
}
```



```
import org.hamcrest.CoreMatchers.equalTo
import org.hamcrest.CoreMatchers.`is`
import org.hamcrest.MatcherAssert.assertThat
import org.junit.Test

class HamcrestAssertionDemo {
    @Test
    fun assertWithHamcrestMatcher() {
        assertThat(sum(2 + 1), `is`(equalTo(3)))
    }
}
```

# JUnit Assumptions

- Assumptions make tests run only under certain conditions

```
import org.junit.Assume
import org.junit.Test

class AssumptionsDemo {
    @Test
    fun testInAllEnvironments() {
        Assume.assumeThat("CI".equals(System.getenv("ENV")),
            () -> {
                // tests performed only when on the CI server
                // ...
            });

        // tests performed in all environments
        // ...
    }
}
```

Source: https://github.com/junit-team/junit4/wiki/Assumptions-with-assume

# @Rule

- Provides additional behavior or state for a test method/class
- Acts like @Before + @After, but reusable once defined
- Common rules:
  - ErrorCollector Rule: allows execution of a test to continue after the first problem is found
  - TemporaryFolder Rule: allows creation of files and folders that are deleted when the test method finishes

# Parameterized Test (1/2)

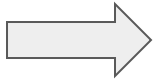
- A parameterized test runs the same test logic **multiple** times with **different inputs and expected outputs**
- **Why use it?**
  - Removes repetitive test code
  - Improves readability and coverage
  - Makes test intent clearer (data-driven testing)

# Parameterized Test (2/2)

```
class Calculator {
          @Test
     fun add_1plus2_returns3() {
                    assertEquals(3, 1 + 2)
     }

     @Test
     fun add_2plus3_returns5() {
         assertEquals(5, 2 + 3)
     }

     @Test
     fun add_5plus7_returns12() {
         assertEquals(12, 5 + 7)
     }
}
```



```
@RunWith(Parameterized::class)
class CalculatorTest(
    private val a: Int,
    private val b: Int,
    private val expected: Int
) {

    companion object {
        @JvmStatic
        @Parameterized.Parameters(name =
"{index}: add({0}, {1}) = {2}")
        fun data(): List<Array<Any>> = listOf(
            arrayOf(1, 2, 3),
            arrayOf(2, 3, 5),
            arrayOf(5, 7, 12)
        )
    }

    @Test
    fun add_returnsExpectedResult() {
        assertEquals(expected, a + b)
    }
}
```

5 mins

# Exercise 2: Parameterized Test

- Write a parametrized test for `getActiveAndCompletedStats`
- There were 3 tests at the end of Exercise 1:
  - e.g.,
    - getActiveAndCompletedStats_two_return50f50f
    - getActiveAndCompletedStats_zero_return0f0f
    - getActiveAndCompletedStats_two_return0f100f
- Integrate 3 tests into a parameterized test with `@RunWith`(Parameterized::class)
  - Refer to the official guide

# JUnit4 Advanced Features

- View official user guide & official docs for more:
  - `@Category`: groups tests into categories for selective test execution
  - `@Suite`: allows grouping multiple tests into test suites

# Now We Know How to Test a Model!